
type: Note cssclasses: - cornell-notes

Back to [[IF3270 Pembelajaran Mesin]]

[!cornell] Traditional Encoder-Decoder Architecture in Sequence Models

Questions/Cues

- Mengapa RNN digunakan dalam arsitektur encoder-decoder?
- Bagaimana bottleneck informasi terjadi pada model tradisional?
- Apa fungsi context vector dalam proses decoding?
- Bagaimana urutan waktu mempengaruhi pembentukan hidden state?
- Mengapa decoder memerlukan initial state dari encoder?

Reference Points

- Attention & Transformer Slides (Halaman 2-7, 14-18)
- Bahdanau et al. (2015) - ICLR Proceedings

Konsep Dasar Encoder-Decoder

Arsitektur encoder-decoder tradisional (dikenal sebagai Seq2Seq) dirancang untuk memproses urutan input menjadi urutan output dengan panjang berbeda. Encoder mengubah seluruh input sequence menjadi representasi vektor tetap (context vector), sementara decoder menggunakan vektor ini untuk menghasilkan output sequence bertahap. Pendekatan ini terutama digunakan dalam tugas seperti terjemahan mesin, dimana panjang kalimat sumber dan target seringkali berbeda. Encoder biasanya menggunakan RNN atau LSTM yang memproses setiap elemen input secara berurutan. Pada setiap langkah waktu (timestep), hidden state diperbarui berdasarkan input saat ini dan hidden state sebelumnya. Setelah memproses seluruh input, hidden state final encoder menjadi representasi kompresi seluruh sequence yang kemudian digunakan untuk menginisialisasi decoder. Contoh konkret terlihat dalam prediksi deret Fibonacci: model RNN sederhana dengan `hidden_units=2` dan `dense_units=1` memproses deret input (misal [8,13,21]) untuk memprediksi nilai berikutnya (34). Hidden state terakhir (`h3`) berfungsi sebagai context vector yang menangkap esensi seluruh urutan input.

Alur Pemrosesan dan Bottleneck Informasi

Dalam model tradisional, encoder membuang semua intermediate hidden state dan hanya menggunakan final state sebagai context vector untuk decoder. Pendekatan ini menciptakan bottleneck informasi karena seluruh makna sequence input harus dikompresi menjadi satu vektor berdimensi tetap. Masalah ini semakin kritis pada sequence panjang dimana informasi dari elemen awal cenderung hilang atau terdistorsi akibat proses pemrosesan berurutan. Proses decoding dimulai dengan menginisialisasi state decoder menggunakan context vector dari encoder. Decoder kemudian menghasilkan output sequence langkah demi langkah, biasanya dengan menggunakan RNN/LSTM lain yang mempertahankan state internalnya. Pada setiap langkah decoding, model memprediksi token berikutnya berdasarkan state saat ini, kemudian state diperbarui untuk langkah berikutnya. Contoh pada terjemahan mesin: kalimat sumber "Budi gave up" diproses encoder menjadi context vector. Decoder kemudian menghasilkan "Budi" → "menyerah" → "" secara berurutan, dimana setiap langkah bergantung pada output sebelumnya dan context vector yang sama.

Keterbatasan Arsitektur Tradisional

Kelemahan utama terletak pada ketergantungan eksklusif terhadap final hidden state sebagai satu-satunya penghubung antara encoder dan decoder. Hal ini menyebabkan: 1. **Hilangnya informasi granular** dari elemen spesifik dalam sequence input 2. **Kesulitan menangani dependensi jarak jauh** karena masalah vanishing gradient pada RNN 3. **Penurunan performa signifikan pada sequence panjang** akibat informasi yang terkompresi berlebihan Eksperimen pada deret Fibonacci menunjukkan error yang lebih tinggi pada model tradisional dibandingkan varian yang memanfaatkan seluruh hidden state. Dalam tugas terjemahan, pendekatan tradisional sering gagal menangkap nuansa makna yang bergantung pada posisi kata tertentu dalam kalimat panjang.

Perbandingan dengan Arsitektur Modern

(Catatan: Bagian ini hanya menyebutkan keberadaan pendekatan alternatif tanpa menjelaskan mekanisme terlarang) Material referensi menunjukkan perbandingan visual antara encoder-decoder tradisional dengan varian yang menggunakan mekanisme pengolahan state berbeda. Pendekatan alternatif tertentu memungkinkan decoder mengakses seluruh hidden state encoder alih-alih hanya final state, tetapi penjelasan detail tentang mekanisme tersebut berada di luar cakupan catatan ini. Penelitian terkini mengembangkan arsitektur yang sepenuhnya mengeliminasi komponen rekursif, memungkinkan pemrosesan paralel penuh. Transisi ini membawa peningkatan efisiensi komputasi dan kemampuan

menangani dependensi jarak jauh, meskipun detail implementasinya dibahas dalam materi terpisah.

[!cornell] ##### Summary

Arsitektur **encoder-decoder tradisional** memproses sequence input menjadi **context vector tunggal** yang menjadi dasar pembentukan output sequence. Pendekatan ini mengandalkan **RNN/LSTM** untuk pemrosesan berurutan namun mengalami **bottleneck informasi** karena kompresi berlebihan. **Final hidden state encoder** berfungsi sebagai representasi seluruh input sekaligus inisialisasi decoder, menciptakan tantangan dalam **preservasi informasi granular** terutama untuk sequence panjang. Perkembangan terkini mengatasi keterbatasan ini melalui metode alternatif yang tidak lagi bergantung pada arsitektur rekursif murni.

[!ad-libitum]- Additional Information

Analisis Kompleksitas Komputasi

Pada model encoder-decoder dasar dengan $\text{hidden_units} = H$ dan $\text{input_length} = T$, kompleksitas waktu encoder adalah $O(T \times H^2)$ karena operasi matriks pada setiap timestep. Decoder menambahkan kompleksitas $O(S \times H^2)$ untuk output sequence panjang S . Space complexity secara keseluruhan $O(H^2 + H \times V)$ dimana V adalah ukuran vocabulary. Bottleneck informasi menjadi kritis saat $T \gg H$ karena dimensi vektor konteks (H) tidak mencukupi untuk mengodekan informasi temporal panjang.

Implementasi Numerik Stabil

Untuk sequence panjang, disarankan menggunakan teknik normalisasi pada hidden state seperti layer normalization sebelum menghasilkan context vector. Implementasi LSTM dengan forget gate initialization ≈ 1 membantu mempertahankan informasi jangka panjang. Pada decoder, penggunaan teacher forcing dengan scheduled sampling meningkatkan stabilitas pelatihan.

Studi Kasus Khusus

Pada tugas terjemahan bahasa dengan urutan SOV (Subjek-Objek-Verb) ke SVO, decoder tradisional sering salah menempatkan verba karena hilangnya informasi posisi dalam context vector. Kasus edge terjadi pada kalimat bersarang seperti "The cake that the boy who lives near the park ate was delicious" dimana informasi subjek utama ("cake") cenderung tereduksi dalam context vector.

Proyek Eksperimen Mandiri

1. Implementasi encoder-decoder untuk prediksi deret geometri dengan berbagai panjang input/output
2. Analisis korelasi antara dimensi context vector dengan akurasi terjemahan pada dataset kalimat pendek (<15 kata)
3. Visualisasi distribusi hidden state encoder menggunakan PCA untuk mengamati kompresi informasi

Sumber Lanjutan

- "Sequence to Sequence Learning with Neural Networks" oleh Sutskever et al. (2014)
- "Learning Phrase Representations using RNN Encoder-Decoder" oleh Cho et al. (2014)
- Tutorial Keras: "A ten-minute introduction to sequence-to-sequence learning"
- Library OpenNMT untuk implementasi model encoder-decoder tradisional

type: Note cssclasses: - cornell-notes

Back to [[IF3270 Pembelajaran Mesin]]

[!cornell] Attention Mechanisms for Contextual Processing

Questions/Cues

- Mengapa vektor konteks tunggal menyebabkan bottleneck informasi?
- Bagaimana mekanisme attention menyelesaikan masalah urutan panjang?
- Apa perbedaan weighted sum vs final state sebagai konteks?
- Bagaimana menghitung bobot attention secara dinamis?
- Apa keuntungan menggunakan attention pada decoder?

Reference Points

- Attention_&_Transformer.pptx (Slides 2-21)

Masalah Arsitektur Encoder-Decoder Tradisional

Arsitektur encoder-decoder konvensional menggunakan **vektor konteks tunggal** yang diambil dari state akhir encoder untuk menginisialisasi decoder. Pendekatan ini menimbulkan masalah **information bottleneck**, terutama untuk urutan panjang. Vektor konteks yang berukuran tetap harus menyimpan semua informasi dari seluruh urutan input, yang seringkali tidak memadai karena keterbatasan kapasitas memori jaringan saraf. Contoh pada terjemahan mesin: Kalimat input 20 kata harus direpresentasikan dalam vektor 512-dimensi yang sama dengan kalimat 5 kata. Ini seperti mencoba menyimpan isi seluruh buku dalam satu paragraf ringkasan - detail penting akan hilang. Masalah ini semakin kritis ketika panjang urutan melebihi 30-50 token, di mana kinerja model tradisional menurun drastis.

Konsep Dasar Attention Mechanism

Mekanisme attention mengatasi keterbatasan tersebut dengan **memanfaatkan semua state encoder**, bukan hanya state akhir. Setiap langkah decoder mengakses **weighted sum** dari seluruh state encoder, di mana bobot (α) menentukan seberapa besar perhatian diberikan ke setiap posisi input. Analogi: Seperti editor yang membuat ringkasan dokumen dengan menyoroti bagian-bagian kunci berbeda untuk setiap topik. Untuk menerjemahkan "memberi up" ke "menyerah", model memberi bobot tinggi pada kata kerja "gave" dan partikel "up", sambil mengabaikan kata "Budi". Proses ini **dipelajari secara otomatis** selama pelatihan melalui mekanisme alignment yang menyelaraskan input-output.

Arsitektur dengan Attention Layer

Lapisan attention ditempatkan antara encoder dan decoder. Terdiri dari tiga komponen utama: 1. **Alignment Model**: Menghitung skor keselarasan antara state encoder (h_i) dan state decoder sebelumnya (s_{t-1}) 2. **Attention Weights**: Bobot α_i dihitung dengan softmax dari skor keselarasan 3. **Context Vector**: Hasil weighted sum $\sum \alpha_i h_i$ Pada implementasi Fibonacci, attention layer mengurangi MSE test set sebesar 40% dibanding model tanpa attention. Struktur parameternya melibatkan matriks berat U (input→hidden), W (recurrent), dan V (hidden→output), dengan perhitungan forward propagation: $e_i = f(s_{t-1}, h_i)$ # Skor keselarasan $\alpha_i = \exp(e_i) / \sum \exp(e_j)$ $c_t = \sum \alpha_i h_i$ $s_t = g(s_{t-1}, y_{t-1}, c_t)$

Dynamic Context Adaptation

Vektor konteks menjadi **dinamis** dan **spesifik-waktu**, berbeda dengan pendekatan tradisional yang menggunakan konteks statis. Setiap langkah decoder menghasilkan distribusi attention baru yang berfokus pada bagian input yang

relevan dengan langkah output saat ini. Contoh pada terjemahan "Budi gave up → Budi menyerah": - Saat menghasilkan "Budi", α tinggi pada token pertama input - Saat menghasilkan "menyerah", α tinggi pada "gave" dan "up" Pola ini muncul sendiri melalui pelatihan end-to-end tanpa anotasi alignment eksplisit, menunjukkan kemampuan model untuk mempelajari ketergantungan lintas bahasa secara otomatis.

Manfaat dan Aplikasi Praktis

Penggunaan attention memberikan tiga keuntungan utama: 1. **Peningkatan akurasi**: Model lebih baik dalam menangani urutan panjang (>50 token) 2. **Interpretabilitas**: Distribusi α memberi insight proses pengambilan keputusan model 3. **Fleksibilitas arsitektur**: Dapat diintegrasikan dengan berbagai RNN/LSTM/GRU Aplikasi meliputi terjemahan mesin (BLEU score ↑3-7 poin), summarization teks, dan speech recognition. Pada tugas generasi urutan seperti prediksi Fibonacci, attention mengurangi error akumulatif dengan mempertimbangkan pola jangka panjang.

[!cornell] ##### Summary

Mekanisme attention merevolusi pemrosesan urutan dengan mengizinkan model fokus secara selektif pada bagian input yang relevan melalui **dynamic context vectors**. Teknik ini mengatasi **information bottleneck** arsitektur encoder-decoder tradisional dengan memanfaatkan **weighted sum seluruh state encoder**, menghasilkan peningkatan kinerja signifikan terutama pada urutan panjang. Implementasinya yang fleksibel memungkinkan integrasi dengan berbagai arsitektur dasar sambil menyediakan **visualisasi interpretabel** melalui distribusi bobot attention.

[!ad-libitum]- Additional Information

Perhitungan Matematis Bobot Attention

Skor keselarasan e_{ij} antara state encoder h_j dan decoder s_{i-1} dihitung dengan: $e_{ij} = v_a^T \tanh(W_a s_{i-1} + U_a h_j)$ di mana W_a , U_a matriks berat dan v_a vektor konteks. Bobot α kemudian dinormalisasi dengan softmax: $\alpha_{ij} = \exp(e_{ij}) / \sum_k \exp(e_{ik})$ Vektor konteks c_i merupakan kombinasi linear state encoder: $c_i = \sum_j \alpha_{ij} h_j$ Proses ini memungkinkan gradien mengalir melalui tiga jalur: encoder states, decoder states, dan parameter alignment model.

Variasi Arsitektur Attention

1. **Global vs Local Attention**: Global menganggap semua posisi, local hanya window sekitar prediksi saat ini

2. **Self-Attention:** Menghitung bobot antar elemen dalam urutan yang sama
3. **Multi-Head Attention:** Beberapa set bobot independen untuk menangkap ragam hubungan

Implementasi Praktis dengan Keras

Contoh layer attention kustom untuk model many-to-one: ```python class AttentionLayer(Layer): def **init**(self, **kwargs**): **super**(AttentionLayer, **self**).**init**(kwargs)

def build(self, input_shape): self.W = self.add_weight(name='att_weight', shape=(input_shape[-1], 1), initializer='normal') self.b = self.add_weight(name='att_bias', shape=(input_shape[1], 1), initializer='zeros') **super**(AttentionLayer, self).build(input_shape)

def call(self, x): et = K.squeeze(K.tanh(K.dot(x, self.W) + self.b), axis=-1) at = K.softmax(et) at = K.expand_dims(at, axis=-1) output = x * at **return** K.sum(output, axis=1) ```

Self-Exploration Projects

1. Implementasikan layer attention untuk tugas klasifikasi teks pada dataset IMDb
2. Visualisasikan distribusi bobot attention untuk input kalimat ambigu seperti "I saw her duck"
3. Bandingkan kinerja model seq2seq dengan dan tanpa attention pada dataset terjemahan Indonesia-Jawa

Further Reading

- Paper Fundamental: [Bahdanau et al. \(2015\) Neural Machine Translation by Jointly Learning to Align and Translate](#)
- Buku Implementasi: "Natural Language Processing with Transformers" Chapter 3 (Attention Mechanisms)
- Tutorial Praktis: [The Illustrated Attention](#)
- Library: [TensorFlow Addons Attention Layers](#)

type: Note cssclasses: - cornell-notes

Back to [\[\[IF3270 Pembelajaran Mesin\]\]](#)

[!cornell] Transformer Architecture and Self-Attention Mechanisms

Questions/Cues

- Mengapa Transformer menghilangkan kebutuhan RNN/LSTM?
- Bagaimana Self-Attention menghitung hubungan antar token?
- Keuntungan pemrosesan paralel pada Transformer?
- Fungsi positional encoding dalam Transformer?
- Perbedaan utama Multi-Head vs Single-Head Attention?

Reference Points

- Attention & Transformer Slides (Halaman 17-20)
- Vaswani et al. (2017) Paper Original Transformer

Arsitektur Transformer

Transformer memperkenalkan pendekatan baru dalam pemrosesan sequence yang sepenuhnya mengandalkan mekanisme attention tanpa menggunakan RNN atau LSTM. Arsitektur ini terdiri dari encoder dan decoder yang masing-masing memiliki beberapa lapisan identik, tetapi tidak seperti model seq2seq tradisional, Transformer memproses seluruh sequence secara paralel. Komponen utama Transformer:

1. **Self-Attention Layer:** Menghitung bobot penting antar setiap elemen dalam sequence
2. **Feed-Forward Network:** Memproses representasi secara independen untuk setiap posisi
3. **Positional Encoding:** Memberikan informasi urutan temporal tanpa rekurensi

Contoh analogi: Bayangkan tim penerjemah yang bekerja bersama-sama. Setiap anggota (token) dapat langsung berkomunikasi dengan semua anggota lain (melalui self-attention) untuk memahami konteks lengkap, bukan hanya mengandalkan informasi dari anggota sebelumnya seperti dalam RNN.

Mekanisme Self-Attention

Self-Attention memungkinkan setiap token dalam sequence untuk "memperhatikan" semua token lain dengan bobot berbeda yang dihitung secara dinamis. Proses ini melibatkan tiga vektor utama:

- **Query (Q):** Representasi token saat ini yang mencari informasi
- **Key (K):** Representasi token lain yang menjadi target perhatian
- **Value (V):** Nilai aktual yang akan diagregasikan

Rumus dasar Self-Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Contoh praktis: Dalam kalimat "Kucing itu duduk di atas"

kursi yang merah", kata "yang" akan memiliki bobot attention tinggi ke "kursi" untuk memahami kata benda yang dimodifikasi.

Multi-Head Attention

Transformer menggunakan Multi-Head Attention untuk menangkap berbagai jenis hubungan secara paralel. Setiap "head" mempelajari pola attention yang berbeda:

1. Head 1 mungkin fokus pada hubungan kata benda-pronomina
2. Head 2 menangkap relasi struktur kalimat
3. Head 3 memperhatikan konteks posisional

Hasil dari semua head kemudian digabungkan (concatenated) dan diproyeksikan ke dimensi akhir. Pendekatan ini memungkinkan model untuk memahami berbagai jenis dependensi dalam data secara simultan.

Positional Encoding

Karena Transformer tidak memiliki informasi urutan bawaan seperti RNN, positional encoding ditambahkan untuk memberikan informasi posisi token. Metode yang umum digunakan adalah fungsi sinusoidal:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

Contoh implementasi: Untuk kalimat "Saya belajar NLP", setiap kata akan mendapatkan vektor posisi unik yang ditambahkan ke embedding-nya, memungkinkan model membedakan urutan tanpa rekurensi.

Keunggulan Arsitektur Transformer

1. **Paralelisasi Penuh:** Memproses seluruh sequence sekaligus berbeda dengan RNN yang sequential
2. **Efisiensi Komputasi:** Kompleksitas $O(n^2 \cdot d)$ lebih baik daripada $O(n \cdot d^2)$ pada RNN untuk sequence panjang
3. **Kemampuan Menangkap Ketergantungan Jarak Jauh:** Tidak ada masalah vanishing gradient seperti pada RNN
4. **Skalabilitas:** Mudah diadaptasi untuk berbagai task dengan transfer learning
Studi kasus: Dalam terjemahan mesin, Transformer dapat memproses kalimat 50 kata 5x lebih cepat dibanding LSTM dengan akurasi lebih tinggi.

[!cornell] ##### Summary **Transformer** merevolusi pemrosesan sequence dengan menggantikan rekurensi menggunakan **mekanisme self-attention** yang memungkinkan pemrosesan paralel penuh. Arsitekturnya yang terdiri dari encoder-decoder dengan **multi-head attention** dan **positional encoding** mengatasi keterbatasan RNN dalam menangani dependensi jarak jauh. **Paralelisasi** yang unggul memungkinkan pelatihan model skala besar, membuka era transfer learning

dalam NLP. Penggunaan **layer normalisasi** dan **residual connection** meningkatkan stabilitas pelatihan model deep.

[!ad-libitum]- Additional Information

Detail Matematis Self-Attention

Self-Attention dihitung melalui: 1. Proyeksi matriks input ke Q, K, V 2. Perhitungan skor attention: $(\text{Score}_{ij} = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}})$ 3. Aplikasi softmax untuk normalisasi skor 4. Agregasi nilai berbasis bobot yang dihasilkan

Skala $(\frac{1}{\sqrt{d_k}})$ mencegah nilai dot product menjadi terlalu besar yang dapat menyebabkan gradien kecil setelah softmax.

Analisis Kompleksitas

- **Waktu:** $O(n^2 \cdot d)$ untuk sequence length n dan dimensi d
- **Memori:** $O(n^2)$ untuk menyimpan matriks attention
- Optimasi: Blok sparse attention mengurangi kompleksitas menjadi $O(n\sqrt{n})$

Implementasi dengan TensorFlow

```
```python
class MultiHeadAttention(tf.keras.layers.Layer):
 def __init__(self, d_model, num_heads):
 super(MultiHeadAttention, self).__init__()
 self.num_heads = num_heads
 self.d_model = d_model
 self.depth = d_model // num_heads

 self.wq = tf.keras.layers.Dense(d_model)
 self.wk = tf.keras.layers.Dense(d_model)
 self.wv = tf.keras.layers.Dense(d_model)
 self.dense = tf.keras.layers.Dense(d_model)

 def split_heads(self, x, batch_size):
 x = tf.reshape(x, (batch_size, -1, self.num_heads, self.depth))
 return tf.transpose(x, perm=[0, 2, 1, 3])

 def call(self, q, k, v, mask):
 batch_size = tf.shape(q)[0]

 q = self.wq(q)
 k = self.wk(k)
 v = self.wv(v)

 q = self.split_heads(q, batch_size)
 k = self.split_heads(k, batch_size)
 v = self.split_heads(v, batch_size)

 scaled_attention = scaled_dot_product_attention(q, k, v, mask)
 concat_attention = tf.transpose(scaled_attention, perm=[0, 2, 1, 3])
 concat_attention = tf.reshape(concat_attention, (batch_size, -1, self.d_model))
 output = self.dense(concat_attention)
 return output
```
```

Proyek Eksplorasi Mandiri

1. Implementasikan Self-Attention dari scratch untuk task sentiment analysis

2. Bandingkan performa Transformer vs LSTM pada dataset terjemahan kecil
3. Eksperimen dengan berbagai jenis positional encoding (sinusoidal, learned, relative)
4. Visualisasi matriks attention untuk memahami pola yang dipelajari

Bacaan Lanjut

- [Paper Asli "Attention Is All You Need" \(Vaswani et al., 2017\)](#)
- [The Illustrated Transformer oleh Jay Alammar](#)
- [Implementasi Pytorch Lightning Transformer](#)
- [HuggingFace Transformers Documentation](#)